

Lecture 1.3: Getting Started with OCaml; Calculator Expressions

Overview of Syllabus

The syllabus can be found [here](#).

Setting up OCaml

Information for setting up OCaml [can be found here](#).

First Steps: Arithmetic Expressions

There are two ways we will run OCaml: via `utop`, OCaml's REPL (Read-Eval-Print-Loop), which allows you to type in commands interactively; and via `ocamlc`, which compiles OCaml to lower-level code that can be executed. We will begin with `utop`.

In `utop`, you can write and evaluate OCaml expressions:

```
utop # 2 + 2;;  
- : int = 4
```

The top line is our input `2+2;;`, while the bottom line shows the result. We must end commands to `utop` with `;;`, because newlines are ignored in OCaml:

```
utop # 2 +  
2;;  
- : int = 4
```

The bottom line shows `utop`'s response. This response contains two things: a **type** (here, `int`), and a resulting value (unsurprisingly, we get `4`). The type `int` is for integers: i.e., whole numbers. OCaml supports addition, multiplication, and integer division through the usual operators:

```
utop # 2 * 3 + 1;;  
- : int = 7
```

Integer division in OCaml (like many other languages) will round down to the nearest integer. You can get the remainder by using the infix operator `mod`:

```
utop # 7 / 3;;  
- : int = 2  
  
utop # 7 mod 3;;  
- : int = 1  
  
utop # (7 / 3) * 3 + (7 mod 3);;  
- : int = 7
```

Exponentiation

Exponentiation in OCaml is not built-in, but we will later be able to *define* it.

To quit `utop`, type CTRL-D, or type `#quit;;`.

Floating-Point Numbers

If you want to not only use whole numbers, you can use the `float` type in OCaml, which denotes **floating-point** numbers. You can think of these as numbers with decimals.

```
utop # 3.2;;  
- : float = 3.2
```

Note that the type for `3.2` is `float`, not `int`. Arithmetic in OCaml is **strongly typed**, which means that arithmetic operators over `int` do not work over `float`. Let's try it out:

```
utop # 3.2 + 5;;  
Error: The constant 3.2 has type float but an expression was expected of type int
```

OCaml gives us a **type error** because `3.2` is a `float`, but `+` expects an `int` on both sides. To fix this, we use floating-point variations of each operator. For addition, that is `+.` . Let's try that out:

```
utop # 3.2 +. 5;;  
Error: The constant 5 has type int but an expression was expected of type float  
Hint: Did you mean 5.?
```

Oops! `+.` expects `float`s on both sides, but `5` has type `int`. The error message we get tells us that we should instead use `5.` , which is the floating-point literal for five:

```
utop # 3.2 +. 5.;;  
- : float = 8.2  
  
utop # 3.2 *. 5.;;  
- : float = 16.  
  
utop # 5.3 /. 5.;;  
- : float = 1.06
```

Booleans

The two booleans in OCaml are written `true` and `false`. The operators between them are `||` (for OR), `&&` (for AND) and `not`:

```
utop # true;;  
- : bool = true  
  
utop # true || false;;  
- : bool = true  
  
utop # true && false;;  
- : bool = false  
  
utop # not true;;  
- : bool = false  
  
utop # not false || true;;  
- : bool = true
```

Note in the last line, `not false || true` gets interpreted as `(not false) || true`. Instead, we could have inserted parentheses to force the other interpretation:

```
utop # not (false || true);;  
- : bool = false
```

Booleans can be inspected using `if .. then .. else ..`:

```
utop # if false then 2 else 3;;  
- : int = 3
```

Strings

Strings can be created by using double quotes:

```
utop # "hi";;
- : string = "hi"
```

Strings can be concatenated together (put side by side) by using the `^` operator (don't confuse it with exponentiation!)

```
utop # "hi " ^ "there";;
- : string = "hi there"
```

Let expressions

Expressions can locally define variables using `let x = .. in ..`:

```
utop # let x = 8 in x / 2 ;;
- : int = 4
```

These can be arbitrarily nested and sequenced:

```
utop # let a = 1 in let b = 2 in a + b;;
- : int = 3

utop # let a = 2 + 2 in 3 * a ;;
- : int = 12
```

Which can be written in a cleaner style by using newlines:

```
utop # let a = 1 in
      let b = 2 in
        a + b;;
- : int = 3
```

Let expressions can also be nested **inside** on the left-hand side of another let expression:

```
utop # let x =
      let a = 1 in a + 1
    in
      x * 2;;
- : int = 4

utop # let x =
      (let a = 1 in a) * (let x = 2 in x + 1) in
      100 / x
    ;;
- : int = 33
```

In `utop`, you can define variables without giving a right-hand side `in ..`:

```
utop # let x = 3;;
val x : int = 3

utop # x + 1;;
- : int = 4
```

The binding of `x` will be available for the rest of the `utop` session. You can update the binding by re-defining it:

```
utop # let x = 3;;
val x : int = 3

utop # let x = 2;;
val x : int = 2

utop # x;;
- : int = 2
```

You can even make reference to the old version of `x` while defining the new one:

```
utop # let x = 3;;
val x : int = 3

utop # let x = x + 1;;
val x : int = 4
```

However, you should **not** think of this as mutating or modifying `x`! Instead, this is **re-defining** a new value of `x` based on the old one.

Equality

Integers can be tested for equality using `=`:

```
utop # let x = 2 + 2 in
      let y = 2 * 2 in
      if x = y then 3 else 4;;
- : int = 3
```

We can use `=` for Booleans as well:

```
utop # false = true;;
- : bool = false
```

The `=` operator will be useful for other types too; we will revisit this later.